Azure Monitor vs Splunk: Simple Guide & Cost Breakdown

A realistic comparison of data collection, costs, and tradeoffs

> **How Monitoring Bills You:** You don't pay based on how many services or virtual machines you have. You pay for **how many Gigabytes (GB) of logs you send**, **how long you keep them**, and **how many times you run searches or alerts** on them.

1. WHAT DATA ARE WE COLLECTING?

Here is what we collect from our application components and third-party systems:

Service Name	What We Send to Monitoring
Web Apps / App Service	Application logs, 4xx/5xx webpage errors, total traffic count, page load time, CPU & Memory usage, slow external calls.
Function Apps	Function runs, code crashes, execution duration, memory consumption, trigger timeouts.
Azure SQL	CPU usage, active connections, long-running queries, database locks, failed logins, storage space left.
PostgreSQL	Active/dropped connections, slow SQL queries, memory usage, disk storage health, database engine error logs.
Cosmos DB	Request Units (RU) spent, 429 rate-limit errors, request delays, database availability, storage growth.
Storage Accounts	Read/Write operations, failed access attempts (403 forbidden), latency times, total data size.
Application Gateway	HTTP status responses, backend server health, website latency, web firewall (WAF) blocks and attacks.
Snowflake	Query execution times, failed database queries, compute warehouse usage, connected user sessions.
Internal APIs	Total API calls, error rates, response times, connection timeouts, dependency call failures.
Cloudflare	CDN traffic, edge errors, blocked malicious requests, cache performance, DNS lookup issues.

2. METRICS VS. LOGS VS. TRACES

Metrics = Numbers
Simple stats like "CPU is at 85%" or "Memory is 4 GB". Very cheap or free to collect. They alert you that *something is slow*.

Logs = Text Lines
Detailed text lines recorded when something happens, like code exceptions. They tell you *why it broke*. Charged per GB.

Traces = Maps
Tracks a single user click as it travels through Web App → API → Database. Shows *exactly where the delay happened*.

3. WHAT CAN (AND CANNOT) BE COLLECTED IN EACH PLATFORM

Feature / Source	Azure Monitor + Log Analytics	Splunk
Azure Resources	**Easy:** Native 1-click connection for all Azure services.	**Requires Effort:** Needs Event Hubs or APIs set up to stream Azure logs to Splunk.
Non-Azure / On-Prem / AWS	**Harder:** Requires installing Azure Arc or special agents on non-Azure servers.	**Easy:** Industry standard. Easily collects logs from on-prem hardware, routers, AWS, etc.
App Tracing (App Insights)	**Automatic:** Shows full code-level trace maps without extra plugins.	**Manual:** Needs Splunk APM tools or manual OpenTelemetry setup in code.
Data You Miss If You Pick Azure	Harder to pull deep network logs from on-premises hardware or non-Azure firewall devices.	N/A (Can collect almost anything if you pay for the ingestion).
Data You Miss If You Pick Splunk	N/A	You might lose out on instant, zero-setup Azure platform metrics unless you set up active polling.

4. AZURE LOG ANALYTICS PLANS & TRADEOFFS

To control Azure costs, you can split your log tables into three different "plans". But saving money limits what you can do with that data:

Table Plan	Ingestion Cost	Query & Alert Capabilities	Best Used For
Analytics	Full Price (~$2.30/GB)	**Full:** Fast search, full KQL query support, instant automated alerts.	App errors, HTTP 500s, DB crashes, active operational issues.
Basic	Low Price (~$0.65/GB)	**Limited:** Billed when you search. No real-time alerts allowed. Simple debugging only.	Debug logs, detailed trace dumps only used when troubleshooting manually.
Auxiliary	Lowest (~$0.15/GB)	**Restricted:** Slow search. Must run search jobs to read data. No alerts.	Long-term compliance logs, audit trails kept for regulatory reasons.

> **Important Tradeoff:** If you put a log table into *Basic* or *Auxiliary* mode to save money, **you cannot attach automated alerts to it**. Only use these tiers for logs you look at after a problem is reported.

5. DATA RETENTION COMPARISON

Platform	Default Free Included Storage	Long-Term / Extended Storage
Azure Log Analytics	31 days for Analytics tables (90 days for App Insights).	Up to 12 years. You pay a small fee (~$0.02 per GB per month) for archived data.
Splunk Cloud	Usually 90 days included in standard cloud contracts.	Additional storage depends on your overall Splunk contract or archive storage limits.

6. REALISTIC COST ESTIMATION COMPARISON

Below is an estimated monthly comparison based on an environment generating **10 GB per day (300 GB / month)** of total telemetry. *(Prices based on standard pay-as-you-go estimated public rates; enterprise discounts will lower these).*

Scenario / Setup	Daily / Monthly Volume	Est. Monthly Cost	Why Cost Differs
Azure Monitor (Collect Everything) All logs sent to Analytics Plan.	10 GB/day (300 GB/month)	**~$690 - $750** / month	Pays standard Analytics ingestion rate ($2.30/GB) on all raw debug & error data.
Azure Monitor (Smart Setup) **4GB Analytics + 6GB Basic/Aux.**	**10 GB/day** **(300 GB/month)**	**~$390 - $440 / month**	**Filters out debug noise. Keeps operational errors on fast search, moves rest to cheap tables.**
Splunk Cloud (Ingest Pricing) 300 GB/month sent to Splunk.	10 GB/day (300 GB/month)	**~$800 - $1,200+** / month	Higher cost per GB due to advanced cross-platform indexing and licensing overhead.
Dual-Logging (Azure + Splunk) Sending full logs to BOTH.	20 GB/day equivalent (600 GB total ingest)	**~$1,500 - $1,900+** / month	**Double Paying!** You pay Azure to collect it, plus Splunk to store and search it.

7. WHICH ONE SHOULD WE CHOOSE?

Your Primary Situation	Clear Recommendation
Most of our application is hosted inside Azure (Web Apps, SQL, Functions).	**Start with Azure Monitor + App Insights.** It is native, cheaper, and faster to configure.
We need to monitor Azure alongside AWS, local physical servers, and company routers.	**Use Splunk.** It connects everything in one single dashboard outside of Azure.
Our Security team already uses Splunk for company-wide threat detection.	Send **only security/audit logs** to Splunk via Event Hub. Keep app performance logs in Azure.
We want to keep monthly cloud bills as low as possible.	**Use Azure Monitor** with table filtering (Analytics for errors, Basic for debug logs).

8. SIMPLE TALKING POINTS FOR YOUR MEETING

"Here is the simple reality on monitoring costs: **We pay for how much log volume we send, not how many Azure services we run.**

Because almost all our components—like Web Apps, Azure SQL, Functions, and PostgreSQL—live in Azure, starting with **Azure Monitor and Application Insights** is our best and most cost-effective option.

To keep costs down, we shouldn't just turn on every debug log by default. We should send our important application errors to Azure's fast search tier (Analytics) so we can alert on them, and send non-urgent diagnostic logs to cheaper basic tiers. This smart filtering cuts our monthly monitoring spend nearly in half.

If we decide we need Splunk for company-wide security later, we should only forward specific security logs there—we shouldn't send identical raw logs to both systems, or we'll be paying twice for the exact same data."

8. ESSENTIAL OPERATIONAL & COST TRACKING KQL QUERIES

Run these real-time KQL queries in Azure Log Analytics to track daily log ingestion costs and troubleshoot live production errors.

8.1 Daily Ingestion Cost Tracker (Find Which Logs Cost the Most)

```
Usage | where TimeGenerated > ago(30d) | summarize IngestedGB = sum(Quantity) / 1000.0 by DataType,
bin(TimeGenerated, 1d) | render columnchart
```

8.2 HTTP 5xx Failure Rate Tracker (Web Apps)

```
AppRequests | where TimeGenerated > ago(15m) | where Success == false | summarize TotalRequests = count(),
FailedRequests = countif(ResultCode startswith "5") by bin(TimeGenerated, 5m), AppRoleName | extend
FailurePercentage = round(100.0 * FailedRequests / TotalRequests, 2) | order by TimeGenerated desc
```

8.3 Application Response Time Percentiles (P95 / P99)

```
AppRequests | where TimeGenerated > ago(1h) | summarize AvgDurationMs = avg(DurationMs), P95DurationMs =
percentile(DurationMs, 95), P99DurationMs = percentile(DurationMs, 99) by bin(TimeGenerated, 5m),
AppRoleName | order by TimeGenerated desc
```

8.4 External API & Dependency Failure Analysis

```
AppDependencies | where TimeGenerated > ago(30m) | where DependencyType == "HTTP" | summarize TotalCalls =
count(), FailedCalls = countif(Success == false), P95LatencyMs = percentile(DurationMs, 95) by Target,
bin(TimeGenerated, 5m) | order by FailedCalls desc
```

8.5 PostgreSQL Critical Server Error Log

```
PGSQLServerLogs | where TimeGenerated > ago(30m) | where errorLevel in ("ERROR", "FATAL", "PANIC") |
project TimeGenerated, LogicalServerName, errorLevel, Message | order by TimeGenerated desc
```

8.6 Cosmos DB Request Unit (RU) Throttling (HTTP 429) & App Gateway Errors

```
CDBDataPlaneRequests | where TimeGenerated > ago(30m) and StatusCode == 429 | summarize ThrottledRequests
= count() by bin(TimeGenerated, 5m), DatabaseName | order by ThrottledRequests desc
```